# 4.2 Analyzing Recursive Algorithms

In Chapter 3, we introduced mathematical techniques for analyzing the efficiency of an algorithm, based upon an estimate of the number of primitive operations that are executed by the algorithm. We use notations such as big-Oh to summarize the relationship between the number of operations and the input size for a problem. In this section, we demonstrate how to perform this type of running-time analysis to recursive algorithms.

With a recursive algorithm, we will account for each operation that is performed based upon the particular ***activation*** of the function that manages the flow of control at the time it is executed. Stated another way, for each invocation of the function, we only account for the number of operations that are performed within the body of that activation. We can then account for the overall number of operations that are executed as part of the recursive algorithm by taking the sum, over all activations, of the number of operations that take place during each individual activation. (As an aside, this is also the way we analyze a nonrecursive function that calls other functions from within its body.)

To demonstrate this style of analysis, we revisit the four recursive algorithms presented in Sections 4.1.1 through 4.1.4: factorial computation, drawing an English ruler, binary search, and computation of the cumulative size of a file system. In general, we may rely on the intuition afforded by a ***recursion trace*** in recognizing how many recursive activations occur, and how the parameterization of each activation can be used to estimate the number of primitive operations that occur within the body of that activation. However, each of these recursive algorithms has a unique structure and form.

### Computing Factorials

It is relatively easy to analyze the efficiency of our function for computing factorials, as described in Section 4.1.1. A sample recursion trace for our factorial function was given in Figure 4.1. To compute factorial(n), we see that there are a total of $n+1$ activations, as the parameter decreases from $n$ in the first call, to $n-1$ in the second call, and so on, until reaching the base case with parameter 0.

It is also clear, given an examination of the function body in Code Fragment 4.1, that each individual activation of factorial executes a constant number of operations. Therefore, we conclude that the overall number of operations for computing factorial(n) is $O(n)$, as there are $n+1$ activations, each of which accounts for $O(1)$ operations.

### Drawing an English Ruler

In analyzing the English ruler application from Section 4.1.2, we consider the fundamental question of how many total lines of output are generated by an initial call to draw_interval(c), where $c$ denotes the center length. This is a reasonable benchmark for the overall efficiency of the algorithm as each line of output is based upon a call to the draw_line utility, and each recursive call to draw_interval with nonzero parameter makes exactly one direct call to draw_line.

Some intuition may be gained by examining the source code and the recursion trace. We know that a call to draw_interval(c) for $c > 0$ spawns two calls to draw_interval(c−1) and a single call to draw_line. We will rely on this intuition to prove the following claim.

**Proposition 4.1:** *For $c \geq 0$, a call to draw_interval(c) results in precisely $2^c - 1$ lines of output.*

**Justification:** We provide a formal proof of this claim by ***induction*** (see Section 3.4.3). In fact, induction is a natural mathematical technique for proving the correctness and efficiency of a recursive process. In the case of the ruler, we note that an application of draw_interval(0) generates no output, and that $2^0 - 1 = 1 - 1 = 0$. This serves as a base case for our claim.

More generally, the number of lines printed by draw_interval(c) is one more than twice the number generated by a call to draw_interval(c−1), as one center line is printed between two such recursive calls. By induction, we have that the number of lines is thus $1 + 2 \cdot (2^{c-1} - 1) = 1 + 2^c - 2 = 2^c - 1$. ■

This proof is indicative of a more mathematically rigorous tool, known as a ***recurrence equation*** that can be used to analyze the running time of a recursive algorithm. That technique is discussed in Section 12.2.4, in the context of recursive sorting algorithms.

### Performing a Binary Search

Considering the running time of the binary search algorithm, as presented in Section 4.1.3, we observe that a constant number of primitive operations are executed at each recursive call of method of a binary search. Hence, the running time is proportional to the number of recursive calls performed. We will show that at most $\lfloor \log n \rfloor + 1$ recursive calls are made during a binary search of a sequence having $n$ elements, leading to the following claim.

**Proposition 4.2:** *The binary search algorithm runs in $O(\log n)$ time for a sorted sequence with $n$ elements.*

**Justification:** To prove this claim, a crucial fact is that with each recursive call the number of candidate entries still to be searched is given by the value

$$\mathsf{high} - \mathsf{low} + 1.$$

Moreover, the number of remaining candidates is reduced by at least one half with each recursive call. Specifically, from the definition of mid, the number of remaining candidates is either

$$(\mathsf{mid} - 1) - \mathsf{low} + 1 = \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor - \mathsf{low} \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}$$

or

$$\mathsf{high} - (\mathsf{mid} + 1) + 1 = \mathsf{high} - \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}.$$

Initially, the number of candidates is $n$; after the first call in a binary search, it is at most $n/2$; after the second call, it is at most $n/4$; and so on. In general, after the $j^{th}$ call in a binary search, the number of candidate entries remaining is at most $n/2^j$. In the worst case (an unsuccessful search), the recursive calls stop when there are no more candidate entries. Hence, the maximum number of recursive calls performed, is the smallest integer $r$ such that

$$\frac{n}{2^r} < 1.$$

In other words (recalling that we omit a logarithm's base when it is 2), $r > \log n$. Thus, we have

$$r = \lfloor \log n \rfloor + 1,$$

which implies that binary search runs in $O(\log n)$ time. ■

## Computing Disk Space Usage

Our final recursive algorithm from Section 4.1 was that for computing the overall disk space usage in a specified portion of a file system. To characterize the "problem size" for our analysis, we let $n$ denote the number of file-system entries in the portion of the file system that is considered. (For example, the file system portrayed in Figure 4.6 has $n = 19$ entries.)

To characterize the cumulative time spent for an initial call to the disk_usage function, we must analyze the total number of recursive invocations that are made, as well as the number of operations that are executed within those invocations.

We begin by showing that there are precisely $n$ recursive invocations of the function, in particular, one for each entry in the relevant portion of the file system. Intuitively, this is because a call to disk_usage for a particular entry $e$ of the file system is only made from within the for loop of Code Fragment 4.5 when processing the entry for the unique directory that contains $e$, and that entry will only be explored once.

To formalize this argument, we can define the ***nesting level*** of each entry such that the entry on which we begin has nesting level 0, entries stored directly within it have nesting level 1, entries stored within those entries have nesting level 2, and so on. We can prove by induction that there is exactly one recursive invocation of disk_usage upon each entry at nesting level $k$. As a base case, when $k = 0$, the only recursive invocation made is the initial one. As the inductive step, once we know there is exactly one recursive invocation for each entry at nesting level $k$, we can claim that there is exactly one invocation for each entry $e$ at nesting level $k$, made within the for loop for the entry at level $k$ that contains $e$.

Having established that there is one recursive call for each entry of the file system, we return to the question of the overall computation time for the algorithm. It would be great if we could argue that we spend $O(1)$ time in any single invocation of the function, but that is not the case. While there are a constant number of steps reflect in the call to os.path.getsize to compute the disk usage directly at that entry, when the entry is a directory, the body of the disk_usage function includes a for loop that iterates over all entries that are contained within that directory. In the worst case, it is possible that one entry includes $n - 1$ others.

Based on this reasoning, we could conclude that there are $O(n)$ recursive calls, each of which runs in $O(n)$ time, leading to an overall running time that is $O(n^2)$. While this upper bound is technically true, it is not a tight upper bound. Remarkably, we can prove the stronger bound that the recursive algorithm for disk_usage completes in $O(n)$ time! The weaker bound was pessimistic because it assumed a worst-case number of entries for each directory. While it is possible that some directories contain a number of entries proportional to $n$, they cannot all contain that many. To prove the stronger claim, we choose to consider the *overall* number of iterations of the for loop across all recursive calls. We claim there are precisely $n - 1$ such iteration of that loop overall. We base this claim on the fact that each iteration of that loop makes a recursive call to disk_usage, and yet we have already concluded that there are a total of $n$ calls to disk_usage (including the original call). We therefore conclude that there are $O(n)$ recursive calls, each of which uses $O(1)$ time outside the loop, and that the *overall* number of operations due to the loop is $O(n)$. Summing all of these bounds, the overall number of operations is $O(n)$.

The argument we have made is more advanced than with the earlier examples of recursion. The idea that we can sometimes get a tighter bound on a series of operations by considering the cumulative effect, rather than assuming that each achieves a worst case is a technique called ***amortization***; we will see a further example of such analysis in Section 5.3. Furthermore, a file system is an implicit example of a data structure known as a ***tree***, and our disk usage algorithm is really a manifestation of a more general algorithm known as a ***tree traversal***. Trees will be the focus of Chapter 8, and our argument about the $O(n)$ running time of the disk usage algorithm will be generalized for tree traversals in Section 8.4.